

Aquí tienes el texto completo del trabajo práctico en formato Markdown, respetando el contenido original de las fuentes e incorporando emojis significativos:

Trabajo Práctico Nº 1: Punteros y gestión de memoria en C

Facultad de Informática

Objetivos de la práctica: Comprender la **asignación de memoria dinámica** y el direccionamiento mediante **punteros** para implementar programas que requieran utilizar grandes estructuras de datos.

1. Punteros

Un puntero es una variable cuyo contenido es una **dirección de memoria**. En la declaración de un puntero se indica el tipo del objeto de datos que se encontrará a partir de la dirección dada. Por ejemplo, la sentencia: `int *p;` declara el puntero `p` que apunta a un objeto de datos de tipo `int`. Para obtener la dirección de memoria de un objeto de datos se agrega el operador `&` delante del identificador del objeto. Ejemplo: `int a = 33; p = &a;` se almacena en `p` la dirección de memoria de la variable `a`, siendo `p` la variable de tipo puntero declarada anteriormente.

Para acceder al objeto de datos apuntado por un puntero, se agrega el operador `*` antes del mismo. Por ejemplo: `int x; x = a + *p;` asigna a la variable `x` el resultado de `a + *p`, es decir, de `a + a`, siendo `a` y `p` las variables anteriormente declaradas.

 **Importante:** El acceso a datos a través de punteros debe hacerse en espacios de memoria pertenecientes a objetos de datos ya declarados por el programa. Ejemplo: `int *y; *y = 10;` en este caso, se declara `y` como un puntero a un entero, y se asigna el valor 10 al objeto de datos apuntado por `y`. La compilación no da errores, pero se encontrarán problemas en la ejecución ya que `y`, en vez de contener una dirección válida de algún objeto de datos ya declarado, contiene un valor indefinido.



Ejercicios:

1.1) Dado el siguiente código en C, resolver: a) Crear, utilizando el editor de texto nano (o vim), el código fuente con el nombre `archivo.c`. b) Compile el programa: `$ gcc archivo.c -o prog`. c) Ejecútelo: `$ ./prog`. d) Razone el resultado.

```
#include <stdio.h>
int main() {
    int x = 10;
    int *p;
    p = &x;
    printf("Dirección de x: %p \n", &x);
    printf("Dirección almacenada en p: %p \n", p);
    printf("Valor de *p: %d \n", *p);
    return 0;
}
```

1.2) Dado el siguiente código en C, resolver: a) Analice el código del programa para determinar el valor final de `a` y `b`. b) Compile el programa y ejecútelo para comprobar los resultados.

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
int main() {
    int a, b;
    int *x, *y;
    a = 10;
    b = 20;
    x = &a;
    y = &b;
    x = y;
    *x = 5;
    return 0;
}
```

2. Punteros y arreglos

El nombre de un arreglo se evalúa a su dirección inicial (`arreglo` es equivalente a `&arreglo`), es decir, es un puntero al inicio del arreglo. De esta manera, es posible operar sobre arreglos utilizando la notación de arreglos o punteros indistintamente. Por ejemplo, dado el arreglo `int a`, el primer elemento puede ser accedido indistintamente como `a` o `*a`, el segundo elemento como `a` o `*(a+1)`, ..., y el décimo como `a` o `*(a+9)`. Lo mismo ocurre a la inversa, es decir, es posible operar con un puntero utilizando la notación de punteros o arreglos indistintamente.

Ejercicios:

2.1) Declare un arreglo `a` de 5 elementos de tipo `float`. Luego asigne cualquier valor al contenido del quinto elemento, accediéndolo a través de la notación de punteros. Finalmente muestre por pantalla el valor del elemento modificado utilizando la notación de arreglos.

2.2) Este ejercicio se basa en descubrir cómo se almacena una matriz en memoria. Resuelva: a) Escriba un programa en C que declare una matriz de 3x4 elementos enteros, y muestre la dirección para cada elemento de la matriz. La salida debe ser: "Elemento[i][j] - Dirección". b) A partir de los valores de direcciones obtenidos para cada elemento, deduzca la disposición de los elementos de la matriz en memoria: almacenamiento por filas o almacenamiento por columnas.

3. Punteros a punteros

Un puntero a puntero es una forma de **indirección múltiple**. Normalmente, un puntero contiene la dirección de una variable objetivo que es de un cierto tipo que no es justamente un puntero. Cuando definimos un puntero a puntero, el primer puntero contiene la dirección del segundo puntero, el cual contiene la dirección de la variable objetivo.

Cuando se desea declarar un puntero a puntero, se ubica un asterisco adicional en el frente de su nombre. Por ejemplo: `int **p2;`

Ejercicio:

3.1) Dado el siguiente código en C. ¿Cuál cree que será el valor final de `num` ? Compile el programa y ejecútelo para comprobar el resultado.

```
#include <stdio.h>
int main() {
    int num;
    int *p1;
    int **p2;
    num = 123;
    p1 = &num;
    p2 = &p1;
    *p1 = num - 23;
    **p2 = *p1 * 2;
    printf("Valor de num = %d\n", num);
    return 0;
}
```

4. Organización de la memoria

Los programas en ejecución (procesos), son organizados o desplegados en segmentos de memoria de diferentes tamaños y con distintas propiedades. Esta organización depende fuertemente del compilador, sistema operativo y hardware subyacente. Un proceso de un programa en C típicamente consiste de los siguientes segmentos:

-  **Stack:** Contiene la pila del programa, una estructura **LIFO** (último que entra primero que sale). Almacena direcciones de retorno, entorno de funciones y variables locales. Su espacio es fijo (normalmente 8MB).
-  **Heap:** Utilizado para la **asignación de memoria dinámica** solicitada en tiempo de ejecución. El programador debe liberarlo explícitamente ya que C no tiene recolector de basura.
-  **Uninitialized Data o BSS:** Contiene variables globales y estáticas no inicializadas o inicializadas en cero.
-  **Initialized Data:** Almacena variables globales o estáticas inicializadas con valores distintos de cero.
-  **Text:** Área que contiene las **instrucciones de máquina** que ejecuta la CPU.

5. Asignación dinámica de memoria

Para almacenar arreglos grandes que exceden el tamaño de los segmentos estáticos, es necesario utilizar el segmento **Heap**.

La gestión se lleva a cabo mediante: * `malloc(n)` : Reserva un bloque de `n` bytes y retorna un puntero al inicio. Si falla, retorna `NULL` . * `free(p)` : Libera el espacio apuntado por `p` . Nunca debe liberarse dos veces un mismo bloque.

Ejercicios:

5.1) Explique qué hace el siguiente fragmento de código:

```
int *p;
p = (int *) malloc(sizeof(int));
*p = 50;
```

5.2) Reemplazar la sentencia `int p` por una declaración con memoria dinámica.

```
int p;
p = 1; p = 2; p = 3; p = 4; p = 5;
```

5.3) Almacenamiento de matrices con memoria dinámica. Es posible utilizar dos estrategias: a) **Almacenamiento con filas separadas:** Un arreglo de punteros `x` donde cada elemento mantiene la dirección de una fila reservada con `malloc` . b) **Almacenamiento con filas unidas:** Se reserva un único bloque contiguo para todas las filas y un arreglo de punteros `x` apunta al inicio de cada fila.

Resolver: Implementar un programa que, utilizando el almacenamiento con **filas unidas**, almacene e imprima la siguiente matriz de 3x4:

```
0 1 2 3
1 2 3 4
2 3 4 5
```

c) Espacio máximo de memoria RAM disponible: i) Utilizando el comando `free`, determinar el tamaño máximo de memoria que podría solicitar un proceso evitando el uso del área de intercambio (swap). ii) Suponga una estructura de datos que almacena una gran matriz de elementos de tipo `double`. Determinar el número máximo aproximado de elementos que podría tener la matriz para el espacio de memoria calculado anteriormente.



Soluciones



Respuestas Explicadas - TP N° 1: Punteros y Gestión de Memoria en C

1 Punteros

Ejercicio 1.1 - Introducción a Punteros

Archivo: `archivo.c`

Análisis:

```
int x = 10;      // Variable entera con valor 10
int *p;         // Declaración de puntero a entero
p = &x;         // p almacena la dirección de memoria de x
```

Explicación: - `&x` → Obtiene la dirección de memoria donde está almacenada `x` -
`p = &x` → El puntero `p` ahora contiene esa dirección - `*p` → Desreferencia el puntero, accediendo al valor almacenado en esa dirección (10)

Resultado esperado:

```
Dirección de x: 0x7fff... (ejemplo)
Dirección almacenada en p: 0x7fff... (misma dirección)
Valor de *p: 10
```

⚠ **Nota:** El código tiene un error en el último `printf` - usa `%p` para imprimir `*p` cuando debería usar `%d`.

Ejercicio 1.2 - Asignación entre Punteros

Archivo: `ej1-2.c`

Análisis paso a paso:

```
a = 10;           // a = 10
b = 20;           // b = 20
x = &a;           // x apunta a a
y = &b;           // y apunta a b
x = y;            // x ahora apunta a b (mismo地址 que y)
*x = 5;           // modifica el valor apuntado por x → b = 5
```

Resultado final: | Variable | Valor | |-----|-----| | a | 10 | | b | 5 |

Conclusión: Al hacer `x = y`, ambos punteros apuntan a `b`. Por lo tanto, `*x = 5` modifica `b`, no `a`.

2 Punteros y Arreglos

Ejercicio 2.1 - Notación de Punteros en Arreglos

Archivo: `ej2-1.c`

Código:

```
float a[5];
*(a+4) = 10.0;           // notación de punteros para acceder al 5º elemento
printf("El quinto elemento es %f\n", a[4]);    // notación de arreglos para imprimir
```

Explicación: - `a` es un puntero al primer elemento del arreglo - `*(a+4)` accede al elemento en la posición 4 (quinto elemento, índice base 0) - Equivale a `a[4]` - El valor asignado es `10.0`

Salida:

```
El quinto elemento es 10.000000
```

Ejercicio 2.2 - Matrices y Almacenamiento en Memoria

Archivo: `ej2-2.c`

Código:

```
int a[3][4];
for(int i = 0; i<3; i++)
    for(int j = 0; j<4; j++)
        printf("a[%d][%d] = %p\n", i, j, &a[i][j]);
```

Conclusión: Las matrices en C se almacenan en **orden row-major (por filas)**. Esto significa que los elementos de una fila se almacenan contiguos en memoria, y luego sigue la siguiente fila.

Ejemplo de salida:

```
a[0][0] = 0x7fff...00
a[0][1] = 0x7fff...04
a[0][2] = 0x7fff...08
a[0][3] = 0x7fff...0C
a[1][0] = 0x7fff...10    ← continúa después de la fila 0
a[1][1] = 0x7fff...14
...
```

Cada `int` ocupa 4 bytes, por eso las direcciones incrementan de 4 en 4.

3 Punteros a Punteros

Ejercicio 3.1 - Indirección Múltiple

Archivo: ej3-1.c

Análisis paso a paso:

```
num = 123;           // num = 123
p1 = &num;           // p1 apunta a num
p2 = &p1;            // p2 apunta a p1 (puntero a puntero)

*p1 = num - 23;      // num = 123 - 23 = 100
**p2 = *p1 * 2;      // **p2 accede a num → num = 100 * 2 = 200
```

Resultado final:

Valor de num = 200

Explicación: - `*p1` accede directamente a `num` - `**p2` es equivalente a `*p1`, que es equivalente a `num` - Ambas operaciones modifican la misma variable `num`

4 Organización de la Memoria

(Ejercicio teórico - no tiene código asociado)

Segmentos de memoria:

Segmento	Descripción
 Stack	Variables locales, parámetros de funciones, direcciones de retorno. LIFO. Tamaño fijo (~8MB).
 Heap	Memoria dinámica (<code>malloc</code> / <code>free</code>). El programador gestiona la liberación.
 BSS	Variables globales/estáticas no inicializadas (inicializadas en 0 por defecto).
 Data	Variables globales/estáticas inicializadas con valor $\neq 0$.
 Text	Código del programa (instrucciones de máquina). Solo lectura.

5 Asignación Dinámica de Memoria

Ejercicio 5.1 - malloc Básico

Archivo: `ej5-1.c`

Código:

```
int *p;
p = (int *) malloc(sizeof(int));
*p = 50;
```

Explicación: 1. `int *p;` → declara un puntero a entero 2. `malloc(sizeof(int))` → reserva 4 bytes en el **Heap** 3. `(int *)` → castea el `void*` retornado por `malloc` a `int*` 4. `*p = 50;` → almacena el valor 50 en la memoria reservada

Importante: El cast es necesario porque `malloc` retorna `void*`. El tipo de casteo determina cómo se interpretará la memoria al recorrerla (de 4 en 4 bytes para `int`, de 1 en 1 para `char`, etc.).

Ejercicio 5.2 - Arreglo Dinámico

Archivo: `ej5-2.c`

⚠ El código original tiene un error de compilación:

```
int p[5];           // ← p declarado como arreglo
int *p = malloc(...); // ← ERROR: redeclaración de p
```

Corrección sugerida:

```
int *p = (int *) malloc(sizeof(int) * 5); // reserva espacio para 5 enteros
p[0] = 1; p[1] = 2; p[2] = 3; p[3] = 4; p[4] = 5;
```

Fórmula para acceso en matrices 1D simulando 2D:

```
elemento[x][y] = p[x * M + y]
```

Donde `M` es la cantidad de columnas.

Ejercicio 5.3 - Matriz Dinámica con Filas Unidas

Archivo: `ej5-3.c`

Implementación con filas unidas (un único bloque contiguo):

```
int X = 3, Y = 4;

// Reservar un único bloque contiguo para todos los elementos
int *a = (int*) malloc(X * Y * sizeof(int));

// Reservar arreglo de punteros para las filas
int **x = (int**) malloc(X * sizeof(int*));

// Hacer que cada fila apunte a la posición correspondiente del bloque
for(int i = 0; i < X; i++){
    *(x+i) = a + i*Y;
}
```

Ventajas de filas unidas: -  Mejor localidad espacial (acceso más rápido por caché)
-  Una sola llamada a `free()` para los datos -  Memoria contigua (mejor para operaciones vectoriales)

Salida del programa:

```
[0][1][2][3]
[1][2][3][4]
[2][3][4][5]
```

Ejercicio 5.3.c) Memoria Máxima Disponible

i) Comando `free` :

```
$ free -h
```

Busca la columna **available** en la fila **Mem**. Ese es el espacio máximo sin usar swap.

ii) Cálculo para matriz de `double` :

```
Tamaño de double = 8 bytes
```

```
Si available = 7.5 GB  $\approx$  8,053,063,680 bytes
```

```
Número máximo de elementos = 8,053,063,680 / 8  $\approx$  1,006,632,960 elementos
```

Esto equivale aproximadamente a una matriz de **31,700 × 31,700** elementos.

Índice de Archivos

Archivo	Ejercicio	Descripción
<code>archivo.c</code>	1.1	Introducción a punteros
<code>ej1-2.c</code>	1.2	Asignación entre punteros
<code>ej2-1.c</code>	2.1	Arreglos y notación de punteros
<code>ej2-2.c</code>	2.2	Matrices y almacenamiento en memoria
<code>ej3-1.c</code>	3.1	Punteros a punteros
<code>ej5-1.c</code>	5.1	malloc básico
<code>ej5-2.c</code>	5.2	Arreglo dinámico (con error)
<code>ej5-3.c</code>	5.3	Matriz dinámica con filas unidas

Trabajo práctico resuelto - Sistemas Paralelos